

UnifECAF
GRADUAÇÃO IA E AUTOMAÇÃO DIGITAL

ROBERT FERNANDO SCHWEPPE

IA Generativa Aplicada ao Desenvolvimento

CURITIBA
2026

ROBERT FERNANDO SCHWEPPE

IA Generativa Aplicada ao Desenvolvimento

Trabalho apresentado à disciplina IA Generativa Aplicada ao Desenvolvimento, como requisito parcial de avaliação da disciplina.

CURITIBA
2026

Sumário

Sumário.....	3
Análise e Discussão.....	4
Contextualização do Problema.....	4
O Cenário Corporativo.....	4
O Caso Aplicado: Serviços ao Cidadão.....	4
Soluções de Referência no Mercado.....	4
Descrição da Solução Desenvolvida.....	4
Visão Geral.....	4
Arquitetura Técnica.....	5
Funcionalidades Implementadas.....	5
RAG (Retrieval-Augmented Generation).....	5
Ferramentas de IA Utilizadas no Desenvolvimento.....	6
Kiro (IDE com IA Integrada).....	6
AWS Bedrock (Google Gemma 3 4b-it).....	6
GitHub Copilot (via Kiro).....	6
MCP Server Playwright (via Kiro).....	6
Como a IA Auxiliou na Criação da Aplicação.....	6
Planejamento Assistido por IA.....	6
Processo de Desenvolvimento.....	7
Exemplos Concretos de Assistência.....	7
Agentes, Automações e Gerenciamento de Contexto.....	8
Gerenciamento de Contexto.....	8
Automações Implementadas.....	8
Agentes Especializados.....	9
Benefícios Obtidos e Limitações Encontradas.....	9
Benefícios.....	9
Limitações.....	9
Aspectos Éticos, Responsabilidade e Governança da IA.....	10
Transparência.....	10
Privacidade e Segurança.....	10
Guardrails e Controle.....	10
Uso Responsável da IA no Desenvolvimento.....	10
Governança.....	10
Apêndice.....	11

Análise e Discussão

Contextualização do Problema

O Cenário Corporativo

Empresas em crescimento acumulam centenas de documentos internos dispersos entre planilhas, PDFs, procedimentos operacionais, políticas corporativas e bases de conhecimento. Embora essas informações existam, elas não estão facilmente acessíveis para os colaboradores no momento em que precisam.

O cenário cotidiano envolve profissionais que perdem tempo procurando respostas para dúvidas recorrentes, consultando colegas por mensagem, navegando em sistemas legados ou buscando documentos antigos em pastas compartilhadas. Esse comportamento gera retrabalho, reduz produtividade e dificulta a disseminação do conhecimento dentro da organização.

O Caso Aplicado: Serviços ao Cidadão

Para demonstrar a solução em um cenário real, o projeto foi aplicado à **Carta de Serviços ao Cidadão das Prefeituras de São José dos Pinhais (PR), Piraquara (PR) e Palhoça (SC)**. As prefeituras disponibilizam centenas de serviços em formato de páginas web com informações sobre requisitos, documentos necessários, prazos, locais de atendimento e horários de funcionamento.

O problema é idêntico ao cenário corporativo: a informação existe, mas o cidadão precisa navegar por múltiplas páginas para encontrar o que precisa. Um assistente inteligente resolve isso ao permitir consultas em linguagem natural como "quais serviços para idoso?" ou "como tirar credencial de estacionamento?".

Soluções de Referência no Mercado

O projeto se inspira em soluções como **Glean, Notion AI e Microsoft Copilot para Microsoft 365**, que utilizam IA generativa para consultar bases de conhecimento corporativas. A diferença é que nossa solução foi construída do zero utilizando IA como copiloto de desenvolvimento, demonstrando como profissionais modernos podem criar produtos digitais de forma acelerada.

Descrição da Solução Desenvolvida

Visão Geral

O **Copiloto Corporativo com IA** é um assistente inteligente que permite consultar bases de conhecimento em linguagem natural. A aplicação possibilita que administradores criem bases, carreguem documentos (arquivos e links web), e disponibilizem essas bases para consulta pública via chat conversacional.

Arquitetura Técnica

A solução utiliza arquitetura **Serverless** na AWS:

```
Usuário (Browser)
  |
CloudFront (CDN + HTTPS)
  |
S3 (Frontend React SPA)

Usuário (Chat/API)
  |
API Gateway (REST + Custom Domain)
  |
AWS Lambda (Node.js 24)
  |
  +--- DynamoDB (5 tabelas: KBs, Chunks, Conversations, Messages, Users)
  +--- S3 (Arquivos originais + context.txt consolidado)
  +--- AWS Bedrock (Google Gemma 3 4b-it - inferência)
```

Funcionalidades Implementadas

Para o usuário final (cidadão/colaborador):

- Chat conversacional em linguagem natural
- Respostas formatadas em Markdown (listas, links, negrito)
- Memória conversacional (histórico mantido na sessão)
- Múltiplas bases de conhecimento disponíveis

Para o administrador:

- Criação e configuração de bases de conhecimento
- Upload de arquivos (PDF, TXT, XLSX, CSV, MD, JSON)
- Import de links web e sitemaps
- Retreino assíncrono com progresso em tempo real
- System prompt e parâmetros do modelo editáveis por base
- Gestão de usuários administradores
- Logs de conversas com filtros e paginação

RAG (Retrieval-Augmented Generation)

O sistema utiliza uma implementação própria de RAG:

- **Indexação:** Ao retreinar, os documentos são parseados (HTML normalizado, texto extraído) e consolidados em um arquivo context.txt no S3
- **Busca:** Quando o usuário faz uma pergunta, um algoritmo de scoring (TF + proximidade) seleciona os 5 trechos mais relevantes

- **Geração:** O contexto selecionado + histórico da conversa são enviados ao modelo Bedrock no formato Messages API
- **Resposta:** O modelo gera uma resposta baseada exclusivamente no contexto fornecido

Ferramentas de IA Utilizadas no Desenvolvimento

Kiro (IDE com IA Integrada)

O desenvolvimento completo do projeto foi realizado utilizando o **Kiro**, um ambiente de desenvolvimento com IA integrada baseado em VS Code. O Kiro foi utilizado como o único copiloto de desenvolvimento durante todo o projeto, realizando:

- Criação da infraestrutura (SAM template, CloudFormation)
- Desenvolvimento do backend (handlers, services, RAG)
- Desenvolvimento do frontend (React, Material UI)
- Configuração de CI/CD (GitHub Actions)
- Debug e resolução de problemas em produção
- Deploys via CLI (AWS, GitHub)

AWS Bedrock (Google Gemma 3 4b-it)

O modelo de IA generativa utilizado na aplicação final é o **Google Gemma 3 4b-it**, acessado via AWS Bedrock. Foi escolhido por:

- Suporte ao formato Messages API (OpenAI-compatible)
- Disponibilidade em us-east-1
- Bom desempenho para instruções em português
- Custo acessível para modelo de 4B parâmetros

GitHub Copilot (via Kiro)

Utilizado de forma indireta através do Kiro como engine de sugestões de código durante a implementação dos componentes.

MCP Server Playwright (via Kiro)

Utilizado para validação da navegação e usabilidade das telas.

Como a IA Auxiliou na Criação da Aplicação

Planejamento Assistido por IA

Antes de escrever uma única linha de código, a IA foi utilizada para criar planos detalhados de execução:

1. **Guia de Execução** ([plans/guia-de-execucao.md](#)): Documento com todas as decisões arquiteturais — tecnologias, padrão de pastas, regras de negócio, formato de URLs, requisitos de cada área. Serviu como "contrato" entre desenvolvedor e IA.
2. **Plano de Implementação** ([plans/implementation_plan.md](#)): Dividiu o projeto em 12 fases independentes e testáveis, cada uma com tabela de etapas e critérios de validação. A IA seguiu esse plano sequencialmente.
3. **Plano de Segurança** ([plans/security_plan.md](#)): Identificou 15+ vulnerabilidades no código gerado (JWT hardcoded, CORS aberto, SSRF, prompt injection) e criou um plano de 11 tasks para corrigi-las.
4. **Análise de Custos** ([plans/costs.md](#)): Documentou o modelo de cobrança de cada recurso AWS, concluindo que a stack parada custa \$0/mês (100% pay-per-use).
5. **Task Tracker** ([plans/task.md](#)): Acompanhamento de progresso com checkboxes para cada sub-tarefa das 12 fases.

Essa abordagem de "planejar primeiro, executar depois" com a IA garantiu que o projeto tivesse direção clara e evitou retrabalho significativo.

Processo de Desenvolvimento

O projeto foi desenvolvido inteiramente através de conversas com a IA no Kiro. O fluxo de trabalho foi:

1. **Planejamento**: Descrição do que deveria ser feito em linguagem natural
2. **Implementação**: A IA gerou o código, criou arquivos, configurou infraestrutura
3. **Deploy**: A IA executou comandos CLI (git, aws, gh) para publicar
4. **Debug**: Quando algo falhava, a IA investigou logs, testou endpoints e corrigiu
5. **Iteração**: Feedback do usuário levou a ajustes incrementais

Exemplos Concretos de Assistência

Infraestrutura completa: A IA criou o template SAM com 5 tabelas DynamoDB, 3 buckets S3, 7 Lambdas, CloudFront, Route53 e API Gateway — tudo a partir de descrições em linguagem natural.

Resolução de bugs em produção: Quando o upload de arquivos retornava erro 400, a IA identificou que o frontend enviava multipart/form-data mas o backend esperava JSON base64. Corrigiu em um único commit.

Otimização de RAG: Quando as respostas não eram relevantes, a IA investigou os dados no S3, identificou que o DynamoDB não paginava (perdendo 284 de 450 chunks), corrigiu e retreinou a base.

Pipeline CI/CD: Criou e ajustou a pipeline sequencial (infra → backend → frontend) com triggers corretos e tratamento de erros.

Agentes, Automações e Gerenciamento de Contexto

Gerenciamento de Contexto

O projeto utiliza gerenciamento de contexto em múltiplos níveis:

No RAG (aplicação):

- Context window do modelo limitada — scoring seleciona apenas trechos relevantes
- Histórico conversacional enviado como multi-turn messages (máx. 10 últimas)
- System prompt configurável por base com guardrails

No desenvolvimento (Kiro):

- Steering files (.kiro/steering/) para manter padrões do projeto
- Task lists para rastrear progresso em tarefas complexas
- Context compaction automático para sessões longas

Automações Implementadas

CI/CD (GitHub Actions):

- Deploy automático sequencial ao fazer push na main
- Validação SAM + build + deploy em cadeia
- Invalidação automática do cache CloudFront

Incrementos via PRs:

- Funcionalidades adicionais planejadas em plans/increment-plan.md
- Cada feature desenvolvida em branch dedicada com PR organizada
- Pipeline executa automaticamente após merge na main

Retreino assíncrono:

- Lambda se auto-invoca com InvocationType Event (fire-and-forget)
- Processamento em batches configuráveis (sequencial ou paralelo)
- Domain blocking: detecta domínios que bloqueiam e pula os restantes
- Cancelamento entre batches via flag no DynamoDB

Import de sitemap:

- Parse de sitemap.xml → extração automática de URLs
- Decodificação de entidades XML
- Deduplicação contra links existentes

Agentes Especializados

O Kiro utilizou sub-agentes internos durante o desenvolvimento:

- **context-gatherer**: Para explorar a codebase antes de fazer alterações
- **semantic-reviewer**: Para revisar mudanças antes de commits

Benefícios Obtidos e Limitações Encontradas

Benefícios

Velocidade de desenvolvimento: Um projeto que levaria semanas com desenvolvimento tradicional foi construído, deployado e iterado em um único dia de trabalho. A IA eliminou a necessidade de consultar documentação de AWS, React, Material UI, DynamoDB — ela já sabia como integrar tudo.

Qualidade de código: O código gerado segue padrões consistentes (TypeScript strict, error handling, tipos bem definidos) sem necessidade de revisão manual de estilo.

Debug acelerado: Problemas que levariam horas para diagnosticar (como a paginação do DynamoDB) foram resolvidos em minutos pela IA que tinha acesso direto aos logs e podia testar hipóteses imediatamente.

Infraestrutura como código: A IA gerou templates CloudFormation complexos (condições, parâmetros, policies IAM) sem erros de sintaxe, algo que exigiria expertise profunda em AWS.

Iteração rápida: Cada feedback do usuário resultava em correção → commit → deploy → verificação em poucos minutos.

Limitações

Modelo Gemma 3 4b-it: Por ser um modelo pequeno (4B parâmetros), apresenta limitações em:

- Alucinação quando o contexto não contém a resposta (inventa informações)
- Dificuldade com busca semântica (não entende sinônimos sem keyword match)
- Janela de contexto limitada

RAG baseado em keyword matching: A busca por relevância usa TF (term frequency) que não captura relações semânticas. Um usuário buscando "idoso" não encontra "Programa Maturidade Ativa" se a palavra "idoso" não aparece no documento.

API Gateway timeout 29s: O retrain de bases grandes excede o limite do API Gateway, exigindo a abordagem assíncrona com auto-invocação de Lambda.

Custo: Uma solução com busca semântica real (embeddings + vector store) teria custo fixo de ~\$350/mês (OpenSearch Serverless), inviável para um projeto acadêmico.

Aspectos Éticos, Responsabilidade e Governança da IA

Transparência

- O chat identifica claramente que a resposta vem de uma IA (nome do agente visível)
- O system prompt instrui o modelo a admitir quando não tem a informação
- O código-fonte é inteiramente aberto no GitHub

Privacidade e Segurança

- Dados sensíveis (JWT secret, chaves AWS) armazenados como GitHub Secrets
- DynamoDB com criptografia SSE (KMS)
- S3 com bucket policy DenyNonSSL + CloudFront OAC
- Rate limiting no chat (previne abuso de custo no Bedrock)
- Sanitização de input contra prompt injection

Guardrails e Controle

- System prompt configurável por base (o administrador define os limites)
- O modelo é instruído a nunca inventar informações fora do contexto
- Guardrail anti-jailbreak no system prompt
- O administrador pode cancelar retreinos em andamento
- Logs de todas as conversas disponíveis para auditoria

Uso Responsável da IA no Desenvolvimento

O projeto demonstra um uso responsável da IA como ferramenta de desenvolvimento:

- A IA foi utilizada como copiloto (assistente), não como substituto do desenvolvedor
- Todas as decisões arquiteturais foram validadas pelo desenvolvedor
- O desenvolvedor manteve supervisão constante sobre o que estava sendo deployado
- Erros da IA foram identificados e corrigidos (ex: formato de request do Bedrock)

Governança

- CI/CD com pipeline sequencial garante que mudanças passem por build + teste antes de deploy
- Separação de ambientes via parâmetro Stage (dev/staging/prod)
- IAM com princípio de mínimo privilégio (cada Lambda tem apenas as permissões necessárias)
- Versionamento completo no Git (histórico de todas as decisões)

Apêndice

A documentação completa está disponível em:

<https://github.com/zastrich/graduacao-10-ia-generativa-aplicada-ao-desenvolvimento>

Vídeo de Apresentação:

<https://youtu.be/4kKg2Lz50dY>

Link do Projeto funcional:

<https://copiloto-corporativo.code200.com.br>